Tareas

Tarea - En proceso

Complete las instrucciones y pulse el botón correspondiente al final de la página.

Título
Tarea 2: TinuC - Microcontrolador RISC-V básico a nivel RTL - 26/27 ENUNCIADO
Entregar
Número de reenvíos permitidos
0
Estado
No ha empezado
Escala de calificaciones
No hay calificación

Instrucciones

TinuC - Microcontrolador RISC-V básico a nivel RTL

Última actualización: 06/11/2026

1. INTRODUCCIÓN

El objetivo general del presente proyecto consiste en la **realización, verificación funcional y validación experimental de un microcontrolador sencillo basado en un subconjunto de la arquitectura del juego de instrucciones del RISC-V**. El microcontrolador debe ser descrito en *SystemVerilog* de modo que sea sintetizable y pueda ser implementado en una FPGA *Cyclone* IV de intelFPGA. Su validación experimental se realizará en el laboratorio mediante una aplicación sencilla propuesta por cada grupo que haga uso de los recursos *hardware* disponibles en el módulo de test. El proyecto abarca, por tanto, los aspectos de verificación funcional, descripción de *hardware* empleando *SystemVerilog*, implementación de un sistema digital integrado en un dispositivo programable complejo y programación en lenguaje ensamblador del RISC-V.

El **número de horas** que **cada alumno** debería dedicar al proyecto es de unas 30h en total, de las cuales 10h serán presenciales y 20h corresponderán a trabajo no presencial.

Se presupone que los alumnos han adquirido (o están en proceso de adquirir) los siguientes conocimientos y competencias:

- Analizar y diseñar circuitos digitales simples (i.e. álgebra de Boole, puertas lógicas, circuitos combinacionales, circuitos secuenciales y máquinas de estados finitos).
- Emplear lenguajes de descripción de hardware para la modelización y síntesis de circuitos combinacionales y secuenciales.
- Programar aplicaciones sencillas en lenguaje ensamblador.
- Diseñar bancos de test que permitan la validación funcional de un módulo mediante lenguajes de descripción de hardware.

El tamaño del grupo será de 4 a 6 personas, pudiendo permitirse grupos de tamaño inferior en situaciones excepcionales y bajo la autorización expresa de los profesores de la asignatura.

2. TEMAS Y OBJETIVOS

Los principales **temas implicados** en el desarrollo del presente trabajo son:

- La arquitectura del juego de instrucciones RISC-V.
- El manejo de instrucciones a nivel hardware: operaciones, representación, soporte de llamadas a procedimientos y modos de direccionamiento.
- La implementación del *datapath* y el *controlpath* en arquitecturas RISC.
- La segmentación (*pipelining*) en procesadores RISC como mecanismo para optimizar el rendimiento.
- La detección y resolución de riesgos de datos y control en arquitecturas segmentadas.

Los aspectos anteriores serán abordados en clase de teoría al ritmo en que vayan siendo necesarios para seguir el plan de trabajo propuesto en la Sección 4.

Los principales **objetivos** formativos de la presente Tarea son:

- Profundizar en el diseño de arquitecturas para procesado de datos y su descripción mediante SystemVerilog.
- Ampliar el dominio de conceptos relacionados con el diseño jerárquico, el particionado y la sincronización de múltiples subsistemas.
- Entender las implicaciones existentes entre el software y el hardware en el marco de una arquitectura RISC.
- Ser capaz de aplicar los conceptos de segmentación (*pipelining*) a una arquitectura de procesado de datos para optimizar su rendimiento.
- Entender los riegos de funcionamiento asociados a la segmentación de un procesador y ser capaz de plantear e implementar estrategias para resolverlos.

3. ENUNCIADO DEL PROYECTO

En el presente proyecto vamos a realizar la implementación de un sencillo microcontrolador al que denominaremos ***TinuC*** basado en un *core* de procesador de 32 bits capaz de ejecutar un subconjunto de la arquitectura del juego de instrucciones RISC-V.

Concretamente, el procesador debe cumplir las siguientes especificaciones:

- 31 registros de propósito general de 32 bits denominados *x1 – x31*, y un registro cableado a 0 denominado *x0*.
- Un registro contador de programa *pc* que almacena la dirección de la instrucción en ejecución.
- Seis formatos de instrucción (R/I/S/SB/U/UJ) de tamaño 32 bits y alineados en fronteras de 4 bytes en memoria de instrucciones.

E implementar **al menos** el siguiente subconjunto del juego de instrucciones RV32I:

- Instrucciones aritmético-lógicas sobre inmediatos enteros: ADDI, SLTI, SLTIU, ANDI, ORI, XORI, LUI, AUIPC, SLLI, SRLI, SRAI
- Instrucciones aritmético-lógicas sobre registros enteros: ADD, SLT, SLTU, AND, OR, XOR, SUB, SLL, SRL, SRA.
- Instrucciones de control de flujo: BEQ, BNE.
- Instrucciones de carga/almacenamiento alineadas: LW, SW.

Como extensión del diseño realizado, se valorará positivamente la incorporación de las siguientes instrucciones adicionales:

- Instrucciones de control de flujo: JAL, JALR, BLT, BLTU, BGE, BGEU.

No será necesario implementar instrucciones de control de hilos ni llamadas al sistema, ni el soporte para el manejo de interrupciones.

La arquitectura del procesador estará **segmentada en 5 etapas** que realizarán las tareas de búsqueda de instrucción (IF), decodificación y lectura del banco de registros (ID), ejecución (EX), acceso a memoria de datos (MEM) y escritura en el banco de registros (WB).

El procesador deberá disponer de memoria de instrucciones (*IMEM*) y de datos (*DMEM*) separadas de 32

kb cada una. Asimismo, el microcontrolador dispondrá de un módulo de entradas/salidas de propósito general (*GPIO*) con un total de 16 pines de entrada y 16 de salida. Un sencillo controlador de memoria interno (*MEM_CTRL*) será el encargado de determinar si el acceso de lectura/escritura se realiza sobre la memoria de datos o sobre el controlador de pines de I/O. Un esquema general del microcontrolador propuesto se muestra en la Fig. 1.

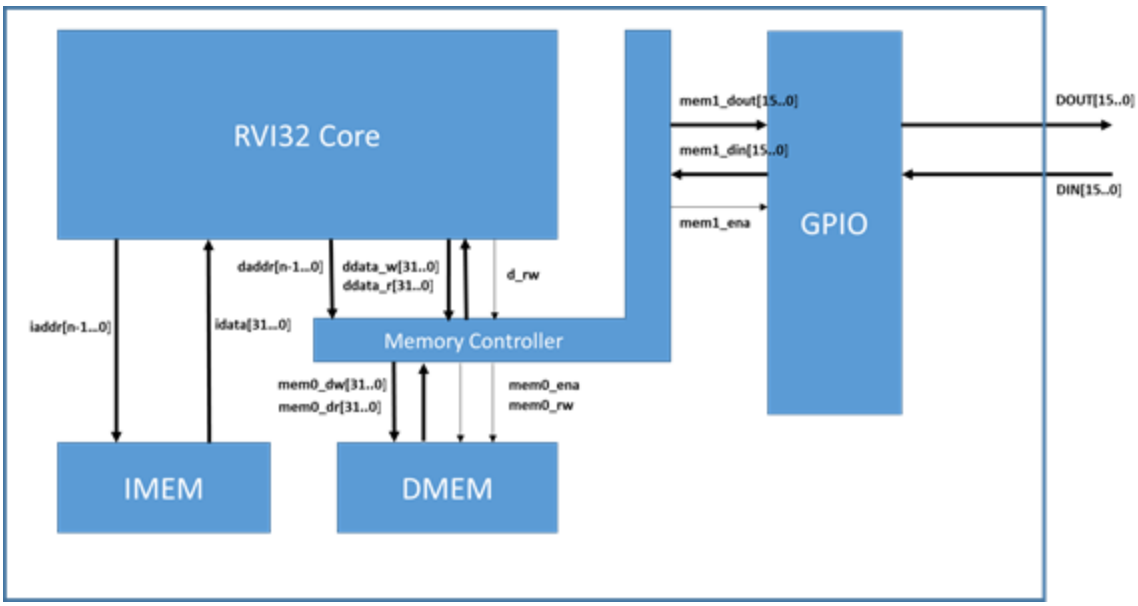


Fig. 1. Esquema general del microcontrolador TinuC. MUY IMPORTANTE: Respetar la denominación de las señales de entrada y salida.

4. PLAN DE TRABAJO

El diseño del microcontrolador se desarrollará en **4 fases** distintas:

- En la **Fase** 1, se llevará a cabo el diseño y validación de las unidades funcionales del procesador: memorias de instrucciones y datos, banco de registros y unidad aritmético-lógica (ALU). Además, se profundizará en el modelo de programación de la arquitectura RISC-V y se realizarán una serie de programas sencillos que luego podrán ser utilizados como *benchmarks* o estímulos de test para el procesador.

- La **Fase 2** consistirá en la implementación de un modelo *single-cycle* del procesador. La prioridad es que dicho modelo sea completamente funcional. El modelo realizado en esta fase deberá ser validado exhaustivamente, y constituirá así mismo un *golden-model*, que permitirá luego llevar a cabo la verificación de la implementación segmentada.

- La **Fase 3** tiene por objeto la segmentación del procesador desarrollado en la fase anterior, así como en la medida de lo posible llevar a cabo la introducción de mecanismos para la detección y el control de riesgos que permitan la ejecución de las instrucciones implementadas sin la necesidad de introducir

huecos de retardo ni realizar detenciones de la segmentación. El procesador resultante deberá ser validado de nuevo utilizando el procesador *single-cycle* como *golden-model*.

- Por último, en la **Fase 4** se abordará el diseño del microcontrolador así como la validación hardware del diseño mediante su implementación en la FPGA Cyclone IV de los módulos de desarrollo disponibles en el laboratorio. El objetivo es realizar un programa que se ejecute sobre el microcontrolador y que, valiéndose de los elementos disponibles en el módulo DE-2, lleve a cabo una aplicación sencilla (e.g. control de LEDs, temporización, uso de los *displays* 7-segmentos,...). El programa realizado debe permitir la interacción con el usuario (i.e. debe utilizar tanto los pines de salida como los de entrada del módulo GPIO).

4.1. DESCRIPCIÓN DE LAS FASES DEL PROYECTO

A continuación se realiza una descripción más detallada de las tareas a desarrollar en cada una de las Fases anteriores:

4.1.1. Fase 1: Unidades funcionales básicas

El objetivo de es desarrollar las principales unidades funcionales del procesador:

- **Memoria de instrucciones y datos**:

Serán necesarias una memoria ROM de 1024 posiciones de 32 bits (i.e. 32 Kbits) para la memoria de instrucciones, y una memoria RAM para el almacenamiento de los datos del programa. La inicialización de los contenidos de la memoria a partir de un fichero de texto puede realizarse utilizando las funciones *$readmemh* y *$readmemb* de Verilog. Puedes encontrar información sobre cómo realizar una descripción correcta de las memorias en el apartado (RECOMMENDED HDL CODING STYLES - INFERRING MEMORY FUNCTIONS) de la guía de recomendaciones de diseño Quartus. Del mismo modo, hay numerosas referencias a las funciones de E/S anteriores, aunque puedes encontrar información y ejemplos en multitud de sitios de internet (por ejemplo, AQUÍ).

En ambos casos, y con el objetivo de realizar una primera implementación *single-cycle* del procesador, la memoria ROM deberá implementar **lecturas asíncronas** y la memoria RAM deberá implementar **lecturas asíncronas** y **escrituras síncronas** (validadas mediante una señal de habilitación de escritura, MemWrite). El interfaz de las memorias (a excepción de la señal de reloj en el caso en que proceda) se muestra en la Fig. 2.

IMPORTANTE: Aunque la realización de lecturas asíncronas impide la implementación de las memorias utilizando los bloques M9K de la Cyclone IV, esta es necesaria para poder llevar a cabo la ejecución de una instrucción en un único ciclo de reloj como haremos en la siguiente. Para una implementación más eficiente en cuanto a la utilización de recursos, podrían realizarse tanto las lecturas como las escrituras de manera síncrona, pero esta modificación tendrá que posponerse a la versión segmentada del procesador (Fase 3).

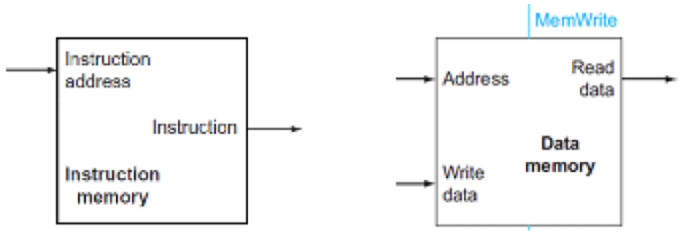


Fig. 2. Memorias de instrucciones y de datos.

- **Unidad aritmético-lógica (ALU):**

La ALU es el bloque funcional de carácter combinacional encargado de llevar a cabo todas las operaciones aritméticas, lógicas, de comparación y desplazamientos necesarias para implementar las diferentes instrucciones de la arquitectura del procesador.

En el caso del RISC-V actúa sobre 2 operandos, y el tamaño de los mismos debe ser igual al de los datos del procesador, es decir 32 bits. Mediante una serie de señales de control se permitirá la selección de la operación a realizar sobre los operandos de entrada. Se ha previsto reservar 4 bits para dar cabida a todas las operaciones necesarias, que en algunos casos pueden ser compartidas por instrucciones distintas (por ejemplo, en el caso de ADDI y ADD la ALU estaría realizando la misma operación, es decir la suma de los operandos de entrada). Un esquema del interfaz de la ALU se muestra en la Fig. 3.

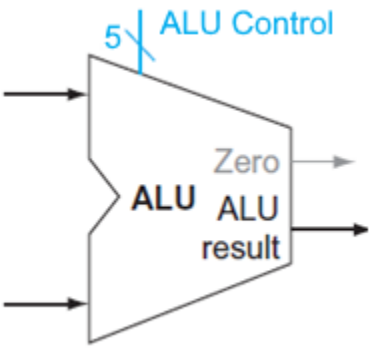


Fig. 3. Unidad aritmético-lógica.

Es habitual incorporar un *flag* de zero que nos servirá para comprobar la condición de igualdad entre operandos en las instrucciones de salto condicional.

- **Banco de registros:**

El banco de registros es el encargado de almacenar y proporcionar acceso a los 32 registros de 32 bits de propósito general del procesador. Dado que las instrucciones del RISC-V pueden tener hasta dos operandos en registros, y almacenar el resultado en un registro destino, es necesaria una estructura capaz de

realizar dos lecturas y una escritura simultáneas sobre el mismo espacio de 32 × 32 bits. El banco de registros deberá implementar **lecturas asíncronas** y **escrituras síncronas** (validadas mediante una señal de habilitación de escritura, RegWrite)

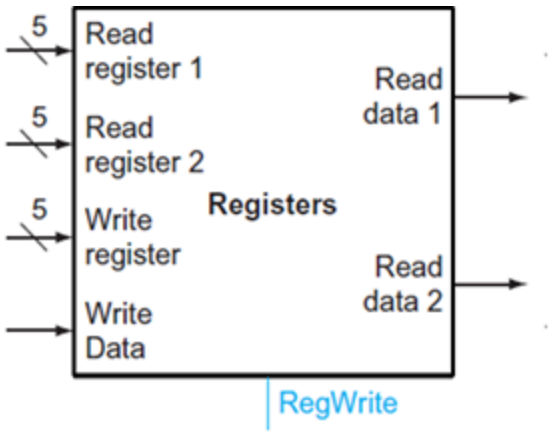


Fig. 4. Banco de registros del procesador RISC-V.

El registro *x0* siempre almacena el valor 0, y cualquier escritura sobre él no debe tener ningún efecto. Un esquema del interfaz del banco de registros se muestra en la Fig. 4.

Sub-Tareas Diseño:

a) Implementar la memoria de instrucciones (ROM) y datos (RAM) del procesador en System Verilog con una profundidad parametrizable (por defecto el tamaño será de 1024 posiciones). El ancho de palabra será fijo y de 32 bits en ambos casos. Las memorias deben poder inicializarse por medio de un fichero de texto que siga la sintaxis requerida por las funciones *$readmemb* y/o *$readmemh*.

b) Implementar la ALU combinacional en System Verilog de manera que soporte todas y cada una de las operaciones aritmético-lógicas asociadas a las instrucciones indicadas en el enunciado. En el caso de instrucciones de carga/almacenamiento y control de flujo la ALU se empleará como sumador para el cálculo de la dirección efectiva de memoria, por lo que no es necesario implementar ninguna función adicional.

c) Implementar el banco de registros del procesador de manera que permita 2 lecturas y 1 escritura simultáneas. Comprobar qué sucede cuando realizamos escritura y lectura sobre un mismo registro (esto puede tener implicaciones en fases posteriores).

Sub-Tareas Verificación:

a) Realizar una validación funcional de las diferentes unidades desarrolladas.

b) Realiza y comprueba, utilizando el simulador Venus (*HTTP://WWW.KVAKIL.ME/VENUS/*) de dos programas sencillos que en posteriores fases serán utilizados para validar el funcionamiento del procesador:

- **Serie de Fibonacci**: Realiza un programa en ensamblador del RISC-V que obtenga y **almacene en memoria de dato**s los N primeros números de la sucesión de Fibonacci (siendo **N una constante almacenada en memoria estática**).
- **Algoritmo de ordenamiento por selección**: Realiza un programa en ensamblador del RISC-V que orde-

ne una serie de números almacenados en memoria estática utilizando el algoritmo de ordenamiento por selección.. Si no recuerdas el algoritmo, puedes encontrar múltiples fuentes de información en Internet sobre CÓMO IMPLEMENTAR EL MISMO EN LENGUAJE C. **Los datos ordenados**, resultado de aplicar el algoritmo, **deberán almacenarse en la memoria de datos.**

Los programas deberán entregarse en ensamblador y en código máquina. Los programas deben utilizar solo instrucciones que se van a implementar en el microprocesador. Los programas no deben utilizar nomenclatura ABI para los registros, es decir, la nomenclatura de los registros debe ser x0, x1, x2, ... x31. Se será estricto en la valoración si esto no se cumple.

4.1.2. Fase 2: Diseño del procesador *single-cycle*

El objetivo de esta fase es desarrollar una versión *single-cycle* del RISC-V. En la Fig. 5 se muestra un esquema simplificado con los diferentes elementos del datapath así como la unidad de control y las señales de control generadas.

Ten en cuenta que en este caso el *controlpath* no es una máquina de estados, sino un decodificador que a partir del código de operación de la instrucción genera las señales de control necesarias para configurar el *datapath* de manera que ésta se ejecute correctamente.

El *datapath* mostrado permite ejecutar instrucciones aritmético-lógicas sobre operandos en registro (R-type) e inmediatos (I-type), así como las instrucciones de carga (I-type), almacenamiento (S-type) y salto condicional (SB-type). Por lo que en nuestro caso deberían poder ejecutarse las instrucciones siguientes: ADDI, SLTI, SLTIU, ANDI, ORI, XORI, SLLI, SRLI, SRAI, LUI, AUIPC, ADD, SLT, SLTU, AND, OR, XOR, SUB, SLL, SRL, SRA, BEQ, BNE, LW y SW.

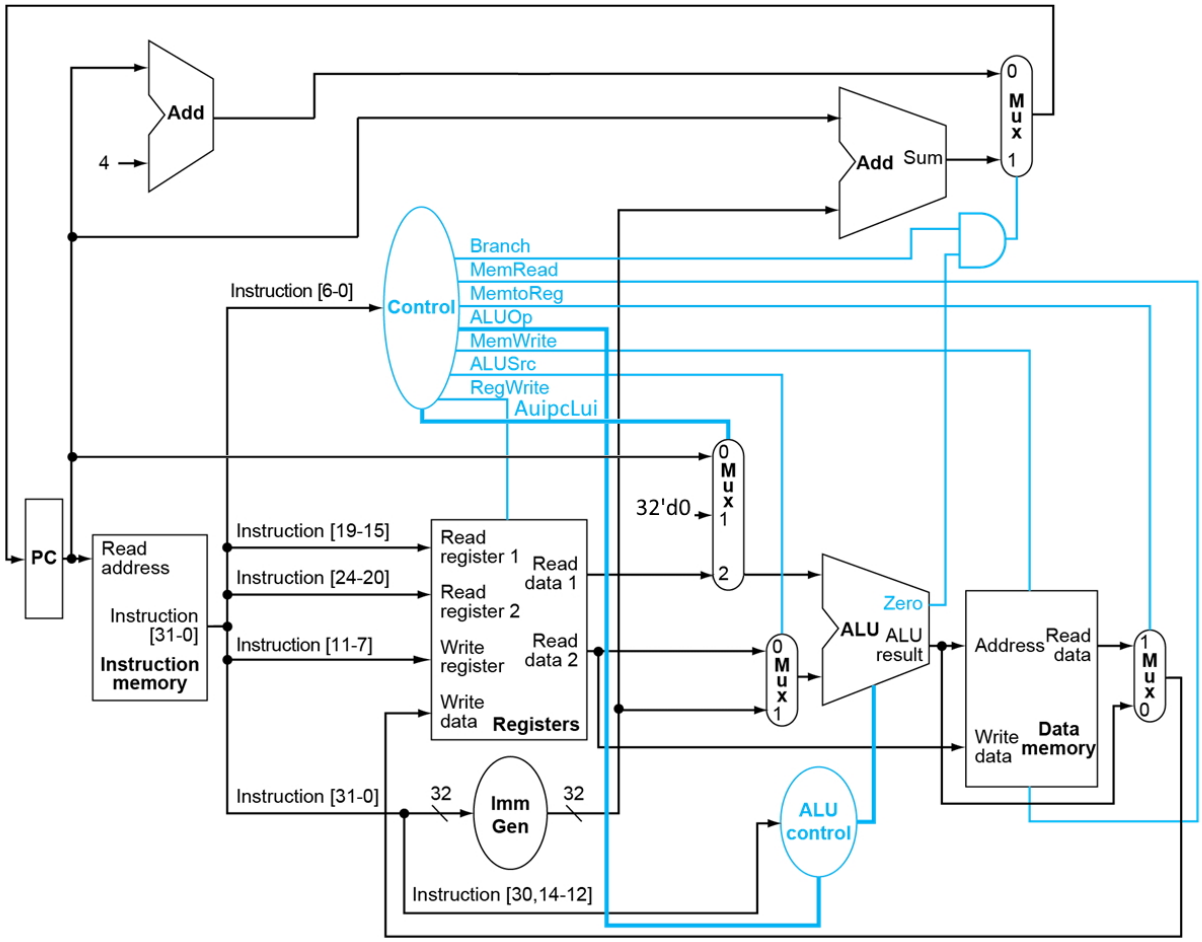


Fig. 5. Datapath (en negro) y controlpath (en azul) del procesador single-cycle.

Sub-Tareas Diseño:

a) Implementar el procesador *single-cycle* a partir del esquema de la Fig.6. Los elementos adicionales (PC, sumadores, multiplexores, generador de inmediato,...) pueden describirse directamente en el módulo *top* del procesador utilizando bloques procedurales *always* y/o asignaciones continuas. Ten en cuenta varios aspectos:

- Las memorias de instrucciones y datos son externas al core del procesador (véase Fig. 1), y por tanto no están incluidas en el módulo *top*.
- El interfaz del procesador debe incluir señales de entrada (reloj, *reset*, bus de datos IMEM, bus de datos de lectura DMEM) y salida (bus de direcciones IMEM, bus de direcciones DMEM, bus de datos de escritura DMEM y señal de lectura/escritura).
- Es necesario respetar la denominación dada a las señales en la Fig. 1, y nombrar a la señal de reloj y reset como CLK y RESET_N respectivamente. De esta manera mantendremos un interfaz compatible entre todos los diseños.

Sub-Tareas Verificación:

a) Verificar funcionalmente (i.e. pre-síntesis) el procesador *single-cycle*. Mediante el uso de los programas desarrollados en la fase 1. Todas las instrucciones implementadas deben ser verificadas mediante uno o varios programas adicionales. Los equipos deberán incluir todos los programas necesarios que verifiquen todas las instrucciones desarrolladas en ensamblador y código máquina. Los programas no deben utilizar nomenclatura ABI para los registros, es decir, la nomenclatura de los registros debe ser x0, xi1, x2, ... x31. Se será estricto en la valoración si esto no se cumple.

b) Cuando el procesador esté verificado con los programas y este sea funcional. Se desarrollará un banco de test que permita validar TODAS las instrucciones implementadas de manera automatizada, es decir, empleando los monitores/chequeadores que consideres oportunos, desarrollando las diferentes competencias en verificación adquiridas en las Tareas 1 y 2. Esta verificación será valorada si la verificación con programas ha sido realizada satisfactoriamente. **ESTE PUNTO ÚNICAMENTE SE VALORARÁ SI EL MICRO ES FUNCIONAL CON LOS PROGRAMAS "SERIE DE FIBONACCI" Y "ORDENAMIENTO POR SELECCIÓN" Y CON LOS PROGRAMAS QUE VERIFICAN TODAS LAS INSTRUCCIONES.**

En las entregas, los alumnos deberán incluir un documento con una tabla donde se listen todas las instrucciones implementadas, qué programa suyo la verifica y en qué línea del código. El documento a rellenar está como adjunto a la tarea.

INSTRUCCIÓN	**VERSIÓN MICRO**	**PROGRAMA**	**LÍNEA QUE LA VERIFICA**
sw	mono	fibonacci.asm	Líneas 25, 35 y 10
addi	segsin	fibonacci.asm	Líneas 3, 5,7, 28
lui	seg	Programa5.asm	Línea 4 y 10
...		....	...

VERSIÓN MICRO: mono: monociclo; segsin: segmentado sin riesgos; seg: Segmentado. Versión del micro donde hemos verificado la instrucción.

IMPORTANTE: ES REQUISITO PARA VALORAR LA FASE 3 y FASE 4 QUE LA FASE 2 SEA FUNCIONAL CON LOS PROGRAMAS "SERIE DE FIBONACCI" Y "ORDENAMIENTO POR SELECCIÓN" Y CON LOS PROGRAMAS QUE VERIFICAN TODAS LAS INSTRUCCIONES.

4.1.3. Fase 3: Implementación segmentada *(pipelined)*

Una vez validada la implementación single-cycle, es posible llevar a cabo la segmentación del *datapath* y el *controlpath* para implementar una *pipeline* de 5 etapas: búsqueda de instrucción (IF), decodificación (ID), ejecución (EX), acceso a memoria (MEM) y post-escritura (WB).

El esquema de la implementación segmentada, partiendo del procesador single-cycle, se muestra en la Fig. 7. Se observa como las señales de control son generadas en la etapa ID y posteriormente distribuidas a los diferentes elementos del *datapath* en la etapa correspondiente.

IMPORTANTE: Hay diversos elementos cuyas lecturas eran asíncronas que ahora admiten lectura síncrona (i.e. salidas registradas). En concreto se trata de las memorias de instrucciones y datos, así como del banco de registros. La implementación síncrona de estos elementos permite al sintetizador inferir el uso de bloques de memoria dedicados de la FPGA, lo que reducirá el área ocupada por el diseño y posiblemente mejorará la frecuencia máxima de operación. Aquellos grupos que así lo consideren pueden modificar los diseños de la Fase 1. Tened en cuenta que si las salidas son registradas no serán necesarios los correspondientes registros de segmentación (para evitar hacer un doble registro de los datos de salida).

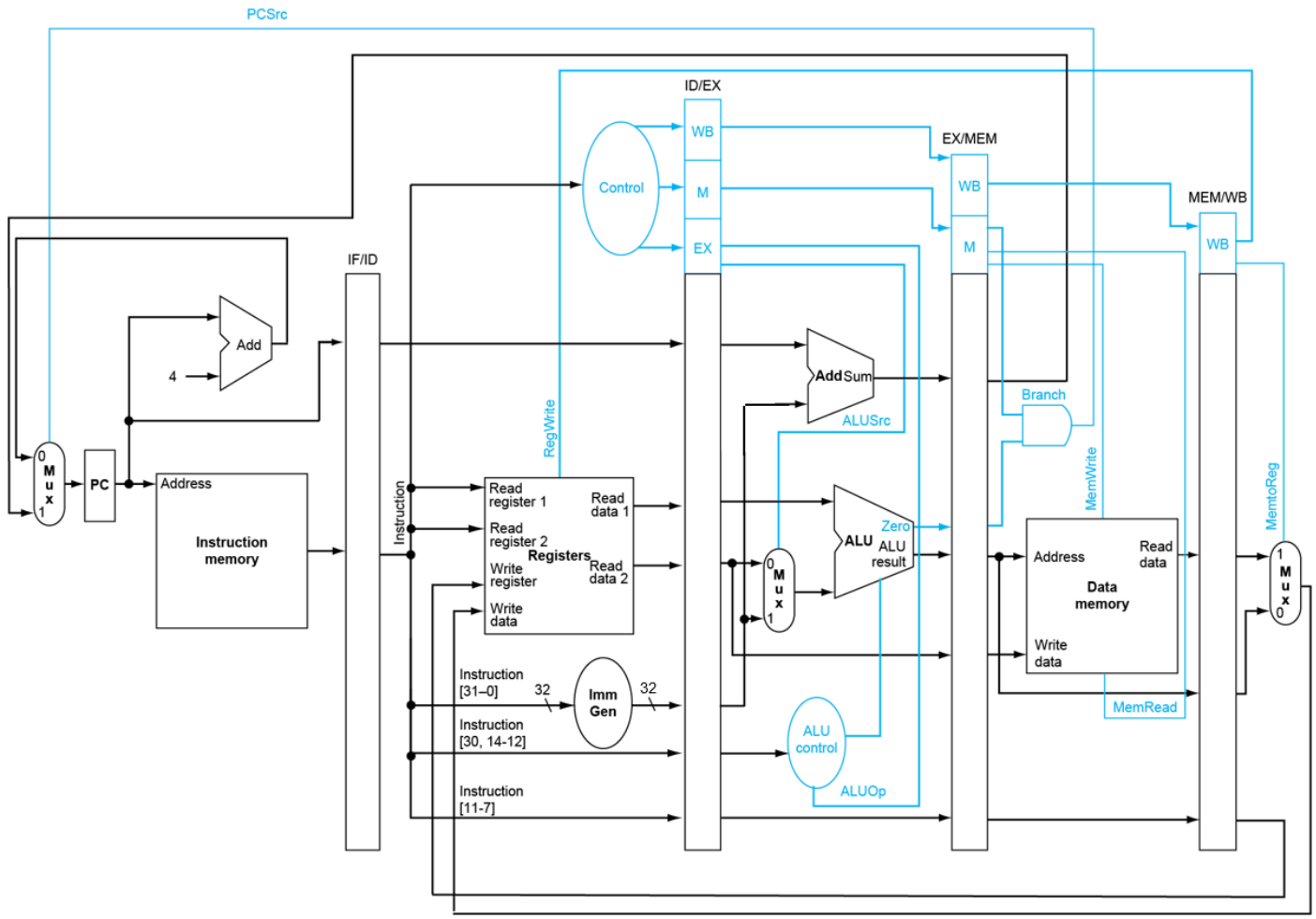


Fig. 7. Datapath (en negro) y controlpath (en azul) del procesador en versión segmentada.

Sub-Tareas Diseño:

a) **Fase 3 –** Implementar la segmentación del procesador partiendo de la versión *single-cycle* desarrollada en la fase anterior, y siguiendo el esquema que se muestra en la Fig. 7. Los registros de segmentación podrán ser descritos directamente en el módulo top mediante bloques procedurales always.

b) **Fase 3.1 – Ampliación del juego de instrucciones**: Modifica el procesador para incluir las instrucciones indicadas en la Sección 3: JAL, JALR, BLT, BLTU, BGE, BGEU.

c) **Fase 3.2 OPCIONAL – Detección y resolución de riesgos de segmentación:** Incorpora una unidad de detección de riesgos así como lo elementos necesarios para poder detectar y resolver, en caso de que sea posible, las situaciones siguientes:

- Riesgos de datos: Riesgos entre instrucciones que puedan ser resueltos mediante adelantamiento de datos (*data-forwarding*) o detención de la segmentación.
- Riesgos de control: Eliminación de instrucciones de la *pipeline* posteriores a un salto que no deban ser ejecutadas.

Sub-Tareas Verificación:

a) **Fase 3 –** Modificar los programas de **Serie de Fibonacci** y **Algoritmo de ordenamiento por selección:,** introduciendo las instrucciones **NOP** necesarias, para que estos sean funcionales en la versión del microprocesador segmentada sin control de riesgos.

b) **Fase 3–** Verificar el microprocesador segmentado sin control de riesgos haciendo uso de los nuevos programas desarrollados.

c) **Fase 3.1** : Crear nuevos programas para verificar las nuevas instrucciones introducidas.

d) **Fase 3.1 y Opcionalmente Fase 3.2 –**Tras verificar el correcto funcionamiento del procesador segmentado mediante uso de los programas, diseñar un testbench que valide el funcionamiento del procesador segmentado utilizando como *golden model* el procesador single-cycle desarrollado en la Fase 2. Si se ha realizado la extensión del juego de instrucciones indicado en la Fase 3.1., será necesario incluirlas también en el modelo *single-cycle*.

e) **Fase 3.1 y Opcionalmente Fase 3.2 –**Realizar una simulación post-síntesis, **incluyendo información de temporización y uso de recursos junto con un análisis comparativo con temporización y recursos con el RISC-V de la fase 2**, utilizando como programas de test los desarrollados en la Fase 1. Validar la correcta ejecución de los mismos.

IMPORTANTE: ES REQUISITO PARA VALORAR LA FASE 3.1 y 3.2. QUE LA FASE 3 SEA FUNCIONAL CON LOS PROGRAMAS "SERIE DE FIBONACCI" Y "ORDENAMIENTO POR SELECCIÓN" SEGMENTADOS. TODAS LAS INSTRUCCIONES NUEVAS DEBEN SER VERIFICADAS MEDIANTE PROGRAMAS COMO MÍNIMO.

4.1.4. Fase 4: Implementación y validación hardware de TinuC

En esta última Fase se pretende realizar una validación experimental del procesador mediante la implementación del microcontrolador en la FPGA del módulo de evaluación DE2-115 del laboratorio. El esquema del diseño se muestra en la Fig. 1.

Para realizar la validación se deberá preparar un programa en ensamblador que emplee alguno de los recursos de la DE2-115 (por ejemplo los LEDs, displays 7-segmentos, switches,...). El objetivo es preparar una sencilla aplicación que demuestre el correcto funcionamiento del microcontrolador.

Sub-Tareas Diseño:

a) Implementar el microcontrolador TinuC siguiendo el esquema que se muestra en la Fig. 1. El módulo GPIO puede estar mapeado en una única posición de memoria inmediatamente a continuación de la memoria de datos. Puedes conectar directamente los pines de entrada y salida a los elementos de la DE2-115 que quieras controlar.

Sub-Tareas Verificación:

a) Desarrolla un programa en ensamblador del RISC-V que, aprovechando los recursos conectados al GPIO, permita demostrar el correcto funcionamiento del microcontrolador. Se trata de preparar un sencillo "*Hola mundo*!" a nivel hardware. Ejemplos de aplicaciones que se pueden implementar son: juegos de iluminación de LEDs, temporizadores con los displays 7-segmentos, calculadora,...

4.2. FLUJO DE TRABAJO

El flujo de trabajo que deberá seguirse para el desarrollo del proyecto se muestra en la Fig. 8. Es importante notar que no podrá pasarse de una fase a la siguiente sin que el diseño esté convenientemente validado.

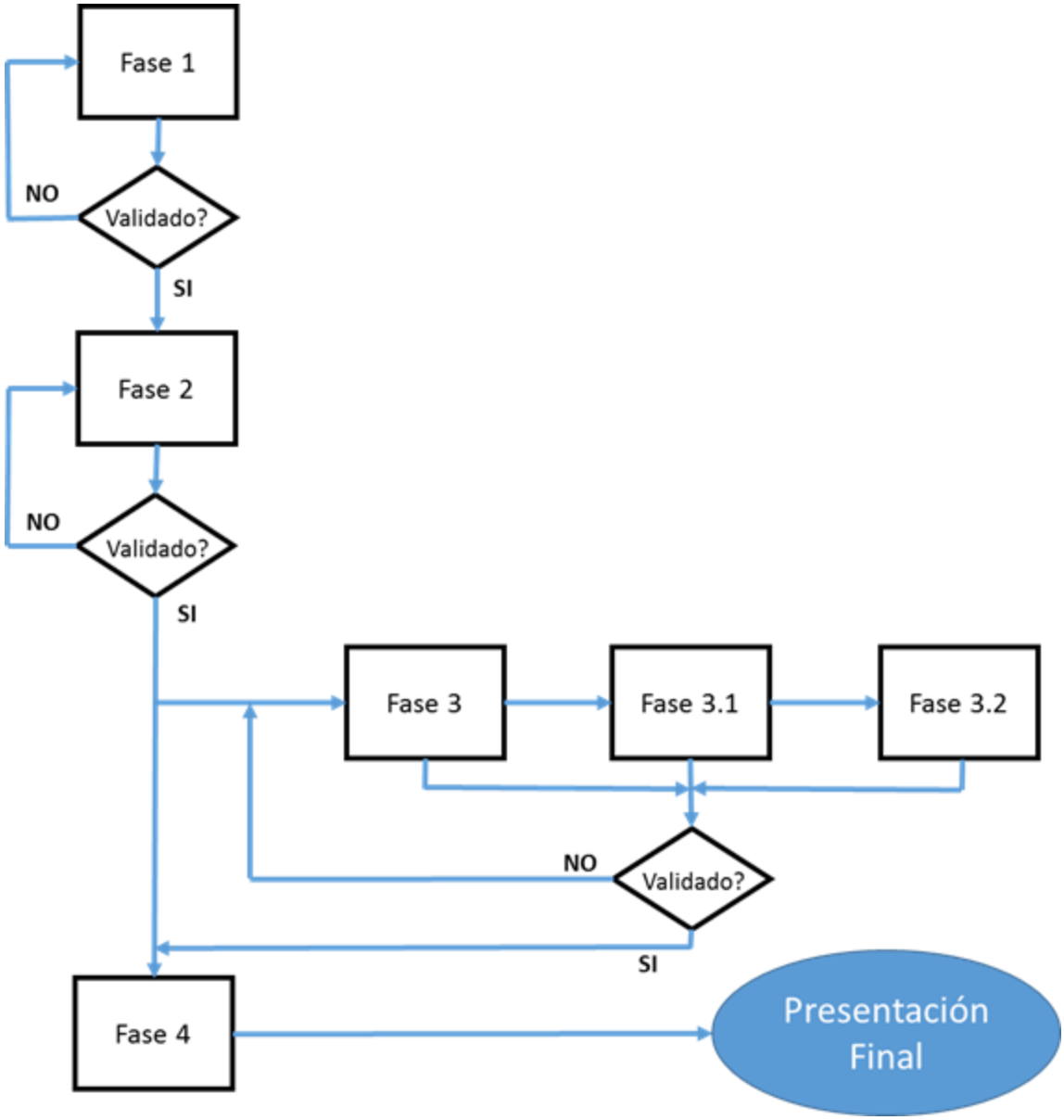


Fig. 8. Flujo de trabajo para el desarrollo del proyecto.

5. ENTREGABLES Y EVALUACIÓN

5.1. ENTREGABLES

Están previstas 2 entregas a lo largo de la realización del proyecto. Las primera entrega se corresponde con Informe de Progreso sobre las Fases 1 y 2 . La última entrega consistirá en toda la documentación sobre el proyecto incluyendo archivos fuente, proyecto Quartus, así como un documento final donde se describa el trabajo realizado así como los resultados obtenidos . Las fases 3, 3.1,3.2 y 4 deberían incluirse en dicho entregable final.

Los detalles sobre las diferentes entregas se muestran en la Tabla I. Los Informes de Progreso tendrán que rellenarse utilizando el documento incluido en los adjuntos a esta Tarea.

IMPORTANTE: TODAS LOS PROGRAMAS USADOS PARA VERIFICAR LOS DISEÑOS DEBEN ESTAR

INCLUIDOS EN LAS ENTREGAS. LOS PROGRAMAS SE SUBIRAN EN ENSAMBLADOR Y EN CÓDIGO MÁQUINA PARA QUE PUEDAN SER EVALUADOS.

Entregable	**Fecha Límite**	**Ubicación**	**¿Quién entrega?**	**Contenidos del entregable**
Informe de Progreso Fase 1 y 2	13/12/26 23h55m	Entrega en tarea	Portavoz del equipo	- Ficheros fuente de diseño y verificación (ZIP-file) - Programas en ensamblador y en código máquina - Documento con la tabla donde se indican todas las instrucciones verificadas, el programa que lo hace y el número de línea de código. - Primer informe de seguimiento firmado
Entrega Tarea **(Fase 3, 3.1., Fase 3.2. y Fase 4)** **FASE 3.2. OPCIONAL**	7/01/27 23h55m	Entrega en tarea	Portavoz del equipo	-Todos los archivos del proyecto (fuentes, Quartus, programas,...) (ZIP-file) -Documento con la tabla donde se indican todas las instrucciones verificadas, el programa que lo hace y el número de línea de código. -Memoria final

Tabla I. Descripción de las entregas del proyecto.

5.2. EVALUACIÓN

Para la evaluación del proyecto tendrá en cuenta tanto el cumplimiento satisfactorio de los diferentes objetivos de diseño y verificación en cada una de las Fases así como su **correcta justificación.**

El alcance de cada una de las Fases en relación al total del proyecto es aproximadamente el siguiente:

- **Fase 1**: 10%
- **Fase 2**: 30%
- **Fase 3**: 35%
- **Fase 4**: 20%
- **Memoria**: 5%

- **La asistencia al laboratorio es obligatoria, por cada falta no justificada se penalizará la nota global de la tarea en un 25%. Los profesores apuntarán la asistencia en cada sesión.**

Importante: Las fases 1 y 2 tienen prevista su entrega el 13 de Diciembre. Cualquier entrega posterior a esa fecha, aunque aceptada a través de los sucesivos reenvíos, tendrá una minusvaloración del 50%.

La valoración se basará en una rúbrica que indicará los diferentes elementos que van a ser evaluados en cada una de las Fases, así como los diferentes grados de cumplimiento. En cualquier caso, los profesores se reservan el derecho a realizar una evaluación global del trabajo realizado.

El profesorado se reserva la posibilidad de modular la nota de los diferentes miembros del equipo de acuerdo, con su grado de implicación en las diferentes partes y alcance de competencias. Para ello, se basarán en las diferentes herramientas para la evaluación continua (informes de progreso, observación diaria etc.).

5. BIBLIOGRAFÍA

1. **“Computer Organization and Design. The HW/SW interface”, David A. Patterson & John L. Henessy, Morgan Kaufmann, (varias ediciones).**

Es el libro de referencia y los esquemas del procesador mostrados están sacados directamente de él. En todas las ediciones hay un capítulo específico que aborda el diseño del procesador y la segmentación, donde se describe en detalle el funcionamiento del mismo.

2. **“Digital Design and Computer Architecture”, David Harris & Sarah Harris, (varias ediciones)**

Es un muy buen libro donde se muestra paso a paso como construir un procesador muy similar (basado en la arquitectura MIPS). Está disponible on-line en la Biblioteca de la UPV.

3. **Material de la asignatura en PoliformaT.**

Recursos adicionales para la tarea

- INFORME_PROGRESO-1-1.DOCX (19 KB; 6 nov 2026 12:03)

- TABLA DE VERIFICACIÓN DE INSTRUCCIONES.DOCX (17 KB; 6 nov 2026 12:03)
- INFORME_PROGRESO-1-1.PDF (148 KB; 6 nov 2026 12:03)
- DESCRIPCION PLACA DE2-115.-2-1-1-2.PDF (10 MB; 6 nov 2026 12:03)
- RISCV_GREENCARD.PDF (10 MB; 6 nov 2026 12:03)
- RUBRICA TAREA 2 RISC-V 2026-2027 V1.PDF (7 KB; 6 nov 2026 14:03)

Envío

Esta tarea es autoevaluable. Debes autoevaluar tu tarea con la siguiente rúbrica antes de enviarla. Debes completar entera la rúbrica para realizar el envío.

TAREA 2 RISC-V 2026-2027 v1

Evaluación de rúbricas

FASE 1 - UNIDADES FUNCIONALES BÁSICAS

FASE 1 - Memoria de instrucciones y datos

Implementar y validar funcionalmente la memoria de instrucciones y datos con una profundidad parametrizable e inicialización desde fichero.

Peso 2 %

No Implementado

El componente no se ha implementado o el diseño es erróneo.

0 Puntos

Implementado

El componente se ha implementado de manera satisfactoria.

(0,14) 7 Puntos

Implementación excelente

El componente se ha implementado de manera excelente (i.e. parametrización, comentarios, buen estilo de descripción,...).

(0,2) 10 Puntos

Comentario para FASE 1 - Memoria de instrucciones y datos

Aceptar

0

FASE 1 - Unidad aritmético-lógica

Implementar y validar funcionalmente la unidad aritmético-lógica.

Peso 2 %

No Implementado

El componente no se ha implementado o el diseño es erróneo.

0 Puntos

Implementado

El componente se ha implementado de manera satisfactoria.

(0,14) 7 Puntos

Implementación excelente

El componente se ha implementado de manera excelente (i.e. parametrización, comentarios, buen estilo de descripción,...).

(0,2) 10 Puntos

Comentario para FASE 1 - Unidad aritmético-lógica

Aceptar

0

FASE 1 - Banco de registros

Implementar y validar funcionalmente el banco de registros del RISC-V permitiendo la realización de 2 lecturas y 1 escritura simultánea.

Peso 3 %

No Implementado

El componente no se ha implementado o el diseño es erróneo.

0 Puntos

Implementado

El componente se ha implementado de manera satisfactoria.

(0,21) 7 Puntos

Implementación excelente

El componente se ha implementado de manera excelente (i.e. parametrización, comentarios, buen estilo de descripción,...).

(0,3) 10 Puntos

Comentario para FASE 1 - Banco de registros

Aceptar

0

FASE 1 - Programación en ensamblador

Desarrollar dos programas sencillos en ensamblador. (Fibonacci y ordenamiento por selección)

Peso 3 %

No Implementado

Los dos programas no se han realizado o son erróneos.

0 Puntos

Implementado

Los programas han sido implementados de manera satisfactoria.

(0,21) 7 Puntos

Implementación excelente

Los programas han sido implementados de manera excelente.

(0,3) 10 Puntos

Comentario para FASE 1 - Programación en ensamblador

Aceptar

0

FASE 2 -DISEÑO Y VALIDACIÓN DEL PROCESADOR SINGLE-CYCLE

FASE 2 - Estructura

Descripción en SystemVerilog del procesador en su versión single-cycle.

Peso 7,5 %

Insuficiente

El procesador no se ha implementado o su estructura es incorrecta.

0 Puntos

Satisfactorio

La estructura del procesador es adecuada y valida funcionalmente, aunque hay algunos aspectos mejorables.

(0,53) 7 Puntos

Excelente

La estructura del procesador es excelente, con un uso adecuado de la jerarquía, una denominación correcta de las señales y un buen estilo de codificación.

(0,75) 10 Puntos

Comentario para FASE 2 - Estructura

Aceptar

0

FASE 2 - Funcionalidad

Funcionalidad incorporada al procesador en su versión single-cycle.

Peso 7,5 %

Insuficiente

El procesador no se ha implementado o su funcionalidad es muy incompleta.

0 Puntos

Satisfactorio

El procesador implementa casi toda la funcionalidad requerida en lo que respecta al juego de instrucciones.

(0,53) 7 Puntos

Excelente

Toda la funcionalidad requerida ha sido correctamente implementada.

(0,75) 10 Puntos

Comentario para FASE 2 - Funcionalidad

Aceptar

0.53
0.53

FASE 2 - Verificación funcional

Funcionalidad incorporada al procesador en su versión single-cycle.

Peso 15 %

Insuficiente

El procesador no se ha validado funcionalmente o ésta validación abarca un número muy limitado de instrucciones.

0 Puntos

Satisfactorio

La validación funcional ha sido realizada como mínimo por medio de los programas de la serie de Fibonacci y de Ordenación por el método de selección. Todas las instrucciones han sido verificadas con su correspondiente programa en ensamblador.

(1,05) 7 Puntos

Excelente

La validación ha sido realizada por medio de un testbench que chequea de manera automatizada el correcto funcionamiento del procesador. ESTE NIVEL NO SE EVALUARÁ SI EL DISEÑO FALLA CON LOS PROGRAMAS FIBONACCI Y ORDENAMIENTO POR SELECCIÓN Y SI NO SE VALIDAN LAS INSTRUCCIONES POR SOFTWARE.

(1,5) 10 Puntos

Comentario para FASE 2 - Verificación funcional

Aceptar

0

FASE 3 - DISEÑO Y VALIDACIÓN DEL PROCESADOR SEGMENTADO

LA FASE 3 NO SE EVALUARÁ SI EL DISEÑO SINGLE CYCLE FALLA CON LOS PROGRAMAS FIBONACCI Y ORDENAMIENTO POR SELECCIÓN.

FASE 3 - Estructura

Descripción en SystemVerilog del procesador en su versión segmentada.

Peso 7,5 %

Insuficiente

El procesador no se ha implementado o su estructura es incorrecta.

0 Puntos

Satisfactorio

La estructura del procesador es adecuada y valida funcionalmente, aunque hay algunos aspectos mejorables.

(0,53) 7 Puntos

Excelente

La estructura del procesador es excelente, con un uso adecuado de la jerarquía, una denominación correcta de las señales y un buen estilo de codificación.

(0,75) 10 Puntos

Comentario para FASE 3 - Estructura

Aceptar

0

FASE 3 - Funcionalidad

Funcionalidad incorporada al procesador en su versión segmentada.

Peso 5 %

Insuficiente

El procesador no se ha implementado o su funcionalidad es muy incompleta.

0 Puntos

Satisfactorio

El procesador implementa casi toda la funcionalidad requerida en lo que respecta al juego de instrucciones.

(0,35) 7 Puntos

Excelente

Toda la funcionalidad requerida ha sido correctamente implementada.

(0,5) 10 Puntos

Comentario para FASE 3 - Funcionalidad

Aceptar

0

FASE3 - Desarrollo de programas versiones sin riesgos

Desarrollo las versiones sin riesgos de los programas en ensamblador del algoritmo de Fibonacci y el de ordenación por selección.

Peso 5 %

Insuficiente

Los dos programas no se han realizado o son erróneos.

0 Puntos

Satisfactorio

Los programas han sido implementados de manera satisfactoria.

(0,35) 7 Puntos

Excelente

Los programas han sido implementados de manera excelente.

(0,5) 10 Puntos

Comentario para FASE3 - Desarrollo de programas versiones sin riesgos

Aceptar

0

FASE3 - Extensión ISA

Inclusión de todas las instrucciones indicadas en la Sección 3 de la Tarea.

Peso 2,5 %

Insuficiente

No ha sido implementado o validado de manera adecuada.

0 Puntos

Satisfactorio

Casi todas las instrucciones adicionales han sido implementadas y validadas con su correspondiente programa en ensamblador

(0,18) 7 Puntos

Excelente

Todas las instrucciones adicionales han sido implementadas y y validadas con su correspondiente programa en ensamblador.

(0,25) 10 Puntos

Comentario para FASE3 - Extensión ISA

Aceptar

0

FASE3 - Verificación funcional

Funcionalidad incorporada al procesador en su versión segmentada.

Peso 15 %

Insuficiente

El procesador no se ha validado funcionalmente o ésta validación abarca un número muy limitado de instrucciones.

0 Puntos

Satisfactorio

La validación funcional ha sido realizada como mínimo por medio de las versiones sin riesgos de los programas de la serie de Fibonacci y de Ordenación por el método de selección. Idealmente , todas las instrucciones deberían verificarse con su correspondiente programa en ensamblador.

(1,05) 7 Puntos

Excelente

La validación ha sido realizada por medio de un testbench que chequea de manera automatizada el correcto funcionamiento del procesador utilizando un "golden-model" (e.g. el procesador single-cycle). ESTE NIVEL NO SE EVALUARÁ SI EL DISEÑO FALLA CON LAS VERSIONES SIN RIESGOS DE LOS PROGRAMAS FIBONACCI Y ORDENAMIENTO POR SELECCIÓN .

(1,5) 10 Puntos

Comentario para FASE3 - Verificación funcional

Aceptar

0

FASE 4 - IMPLEMENTACIÓN Y VALIDACIÓN HARDWARE

LA FASE 4 NO SE EVALUARÁ SI EL DISEÑO FALLA CON LOS PROGRAMAS FIBONACCI Y ORDENAMIENTO POR SELECCIÓN.

FASE 4 - Implementación de TinuC

Implementar el microcontrolador incluyendo el módulo GPIO.

Peso 10 %

Insuficiente

No ha sido implementado o de manera muy limitada.

0 Puntos

Satisfactorio

Ha sido implementado de manera adecuada.

(0,7) 7 Puntos

Excelente

Ha sido implementado de manera excelente (e.g. descripción del top del microcontrolador adecuada, correcto mapeo en memoria e incluyendo un GPIO con entradas y salidas).

(1) 10 Puntos

Comentario para FASE 4 - Implementación de TinuC

Aceptar

0

FASE 4 - Verificación hardware de TinuC

Verificar el funcionamiento del procesador mediante un programa sencillo en el módulo DE2-115

Peso 10 %

Insuficiente

No se ha validado en el módulo DE2-115.

0 Puntos

Satisfactorio

Se ha validado en el módulo DE2-115 de manera sencilla (e.g. encendido y apagado software de un LED).

(0,7) 7 Puntos

Excelente

Se ha validado de manera elegante en el módulo DE2-115, utilizando diversos elementos del mismo.

(1) 10 Puntos

Comentario para FASE 4 - Verificación hardware de TinuC

Aceptar

0

PRESENTACIÓN DE LA MEMORIA Y RESULTADOS DE LA TAREA

Memoria y resultados.

Presentación de la memoria y trabajo realizado por parte del grupo.

Peso 5 %

Insuficiente

La presentación ha sido insuficiente.

0 Puntos

Satisfactorio

La presentación ha sido adecuada.

(0,35) 7 Puntos

Excelente

La presentación ha sido excelente.

(0,5) 10 Puntos

Comentario para Memoria y resultados.

Aceptar

0

Total: **1,53**

Texto de la tarea

Esta tarea permite envíos usando el editor de texto y también subiendo archivos adjuntos. Introduzca su texto en el siguiente editor y/o use el botón de Examinar o de "Seleccionar archivos" para incluir otros documentos. **Le recomendamos que guarde frecuentemente su trabajo antes de enviar la tarea**.

Formato | Fuente HTML | Estilo | Fuente | Tamaño

Palabras: 0, Carácteres (con HTML): 0/1000000

Adjuntos

No hay adjuntos todavía

Seleccionar un archivo del equipo Examinar... No se ha seleccionado ningún archivo. o selecciona archivos de 'Mi Sitio' o un sitio

Continuar envío | Vista preliminar | Guardar borrador | Cancelar

No olvide guardar o continuar

Enlace

- **Tareas**

  Acortar URL

https://poliformat.upv.es/portal/directtool/e23ee53f-81c6-4337-b0b3-a6f2f013c512/